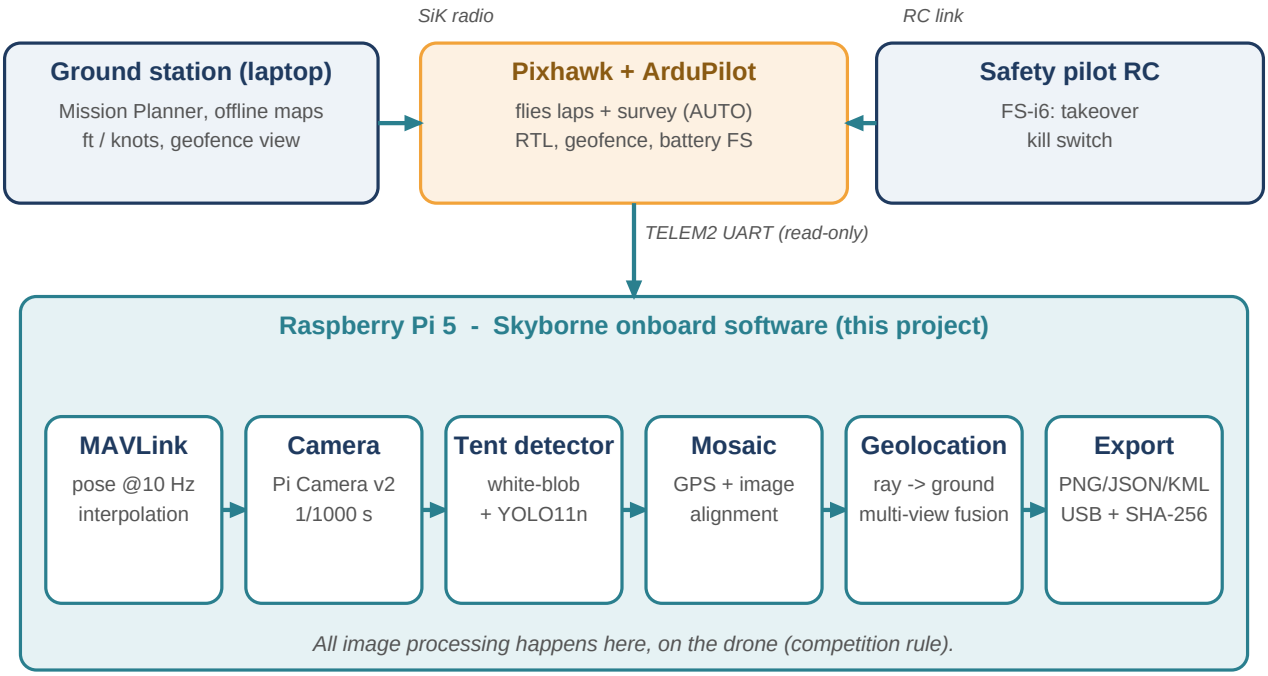
Skyborne NEO'26

Onboard Perception & Mapping Software

How the complete software works, how it was tested, and how to run it



Competition task	What our software does	Status
Endurance laps	ArduPilot AUTO mission (tools/make_mission.py creates laps + DO_JUMP x5). The Pi only listens.	Mission generator done
Risk mapping	Photos are placed on a north-up map during flight; final_map.png on USB before mission time ends.	Working in simulation
Target detection	White tent detected onboard, box drawn on the map, GPS printed with a 95 % error radius.	Working in simulation
All processing onboard	Everything runs on the Raspberry Pi 5 CPU; no ground computer.	Designed for Pi 5

Perception & autonomy software - Team Skyborne. Generated from the code and the simulation results in this repository.

1. The big picture

The drone has two computers with completely separate jobs. The **Pixhawk** (ArduPilot) flies: waypoints, laps, survey pattern, return-to-launch, geofence and battery failsafes. The **Raspberry Pi 5** sees: it reads where the drone is, takes photos, builds the map, finds the tent and writes the USB stick. The Pi software is **read-only** towards the flight controller. It never arms, changes mode or steers, so a software crash on the Pi cannot crash the drone.

Every photo goes through the same six steps. The map and the tent list are built **during the flight**, so very little is left to do after landing:

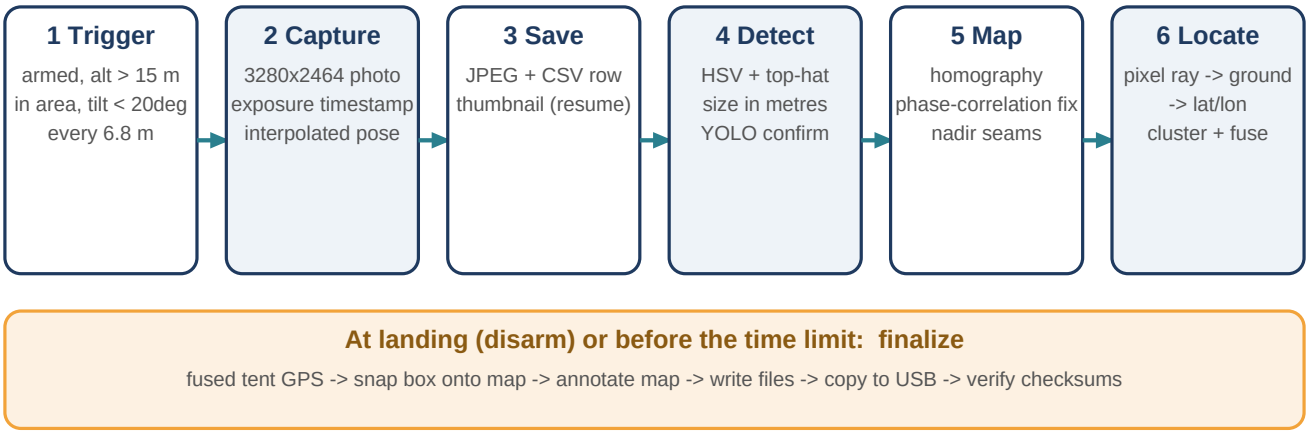


Figure 1 - what happens for each photo and at the end of the mission.

Why this design

- **Map built in flight, not after.** Full photogrammetry (e.g. OpenDroneMap) needs 30-60 min on a laptop. Our map is finished seconds after landing, which matters because every second over time costs 0.5 %.
- **Geometry first, AI second.** We know the altitude, so every pixel has a real size in metres. A tent is 2-20 m^2, a car roof has dark windows, a building is 100 m^2. These physical rules are fast, explainable in the report, and need no training data.
- **Many views per tent.** With 75 % forward and 40 % side overlap the tent appears in 7-8 photos. Fusing them lowers the error and stops one false detection from becoming the answer.
- **Same code everywhere.** Simulation, SITL, replay and real flight run the identical pipeline (pipeline.py). Only the source of photos and positions changes, so what we test is what flies.

2. The software modules

Folder / file	Responsibility
config/settings.py	Every number in one place (camera, overlap, thresholds, time limit). Override in config/user_settings.json without touching code.
communication/mavlink_reader.py	Background thread: connects to the Pixhawk, requests GLOBAL_POSITION_INT, ATTITUDE, GPS_RAW_INT, SYS_STATUS, DISTANCE_SENSOR at 10 Hz, auto-reconnects, logs mission_log.csv, sends STATUSTEXT to Mission Planner.
communication/telemetry.py	Pose type + buffer that interpolates position/attitude to the exact exposure time of a photo (yaw along the shortest arc).
camera/camera_capture.py	Picamera2 back-end (full 8 MP, fixed 1/1000 s shutter, sensor timestamp -> monotonic clock) and USB-webcam back-end for bench tests.
detection/tent_detector.py	White-blob detector with real-world size rules + fusion with YOLO.
detection/yolo_detector.py	YOLO11n/YOLOv8n ONNX inference (onnxruntime or OpenCV DNN), optional tiling for high altitude.
mapping/camera_model.py	Pinhole + distortion model, roll/pitch/yaw rotations, pixel <-> ground projection.
mapping/image_mapper.py	Incremental mosaic: homography placement, phase-correlation alignment with GPS anchoring, exposure matching, nadir seams.
mapping/geolocator.py	Sightings -> clusters -> fused tent position + 95 % error radius.
output/map_renderer.py	Final map: tent box, GPS label, close-up inset, north arrow, scale bar, info panel.
output/export_results.py	Writes products, SHA-256 manifest, finds/mounts the USB stick, copies and verifies.
pipeline.py	The mission: capture gate, per-photo processing, resume after reboot, finalize.
main.py	Entry point: sim | sitl | flight | replay | detect.
simulation/	Synthetic world with tent + decoys, simulated flight with sensor errors, simulated cameras (also driven by SITL).
tools/	Mission generator, camera calibration, YOLO dataset generator, YOLO training (script + Colab notebook).
deploy/	Pi setup script, systemd autostart service, ArduPilot parameter reference.

3. Telemetry and photo capture

3.1 Reading the flight controller (MAVLink)

The Pi is wired to the Pixhawk's TELEM2 port (UART, 921600 baud, MAVLink 2). A background thread receives messages continuously and stores timestamped samples:

Data	MAVLink message	Used for
Latitude, longitude, relative altitude	GLOBAL_POSITION_INT	photo position, height H
Roll, pitch, yaw	ATTITUDE	photo orientation (map + tent GPS)
GPS fix type, satellites, HDOP	GPS_RAW_INT	capture only with a 3D fix
Battery voltage / %	SYS_STATUS	logged; shown on the map metadata
Armed state, flight mode	HEARTBEAT	start clock at arm, finalize at disarm
Down rangefinder (optional)	DISTANCE_SENSOR	true height above ground

Timing matters. At 5 m/s the drone moves 0.5 m between two 10 Hz messages. The camera returns the sensor's exposure timestamp. We convert it to the same clock as the telemetry and **interpolate** the pose at that instant, which removes up to half a metre of tagging error.

3.2 When is a photo taken?

- armed, 3D GPS fix, altitude above 15 m (no photos during take-off/landing);
- tilt under 20 deg (skips steep turns, which give oblique, distorted photos);
- inside the survey polygon (+15 m margin), so no time is wasted on the lap course;
- moved at least footprint x (1 - 75 % overlap) = **6.8 m at 30 m altitude** (about every 1.4 s at 5 m/s), and at least 0.8 s since the last photo (Pi processing budget).

3.3 Camera numbers (Pi Camera Module v2, IMX219)

Quantity	Value at 30 m	Note
Resolution	3280 x 2464 (8 MP)	full sensor
Focal length	3.04 mm = 2714 px	pixel pitch 1.12 um
Ground sample distance	1.11 cm / pixel	matches the project brief
Footprint	36.3 m x 27.2 m	across x along track
Tent (2.4 x 2.1 m)	~215 x 190 px	easily resolvable
Shutter	1/1000 s	motion blur 0.5 cm at 5 m/s

4. Finding the white tent

We use a **two-stage hybrid detector**. Stage 1 always runs and needs no training. Stage 2 is a neural network that switches on automatically when models/tent_yolo.onnx exists.

4.1 Stage 1 - white-blob detector with physical rules

- **White mask (HSV):** saturation low and brightness above an *adaptive* threshold (median + 0.55 x (99.9th percentile - median)). The threshold follows sun, cloud and auto-exposure, and grey concrete (V~170) falls below it.
- **Brighter than its surroundings:** we subtract a local background (a grey-level opening with a kernel larger than the biggest tent, computed at 1/8 scale for speed). Large bright areas such as dry ground are removed. *Lesson from testing: our first version used a fixed small kernel that erased the middle of the tent and missed 7 of 8 views. Sizing it from metres fixed that.*
- **Real-world size:** GSD = H / f. Each blob gets an area in m^2 and side lengths in metres. Tent: 2.5-22 m^2, sides 1.2-6 m, aspect ratio up to 2.2.
- **Shape and content:** solidity (convex, compact), and the fraction of dark pixels inside, which rejects white cars because of their windows.
- Each rule gives a 0-1 score; confidence = geometric mean. Boxes touching the image edge are down-weighted.

Object in the test world	Why it is rejected
White car 4.5 x 1.8 m	dark windows inside, aspect ratio 2.5, low solidity
White-roof building 12 x 8 m	area 96 m^2 > 22 m^2
Concrete pad	brightness below the adaptive white threshold
Road markings 3 x 0.15 m	too thin (short side 0.15 m < 1.2 m)
White stones	area far too small
Orange tent (same size!)	saturation too high - not white

4.2 Stage 2 - YOLO11n (why this model)

Model	Params	Pi 5 CPU / frame (640 px)	Verdict
YOLO11n / YOLOv8n	2.6-3.2 M	~0.25-0.5 s	**Chosen**: fits 1 frame/s, good on small objects
YOLO11s	9.4 M	~1 s	backup if more accuracy needed
RT-DETR-R18 (transformer)	20 M	several s	too slow on the Pi CPU
DETR / ViT detectors	40 M+	far too slow	need a GPU

Timings are typical published figures for these model sizes, not measured on our Pi yet. We will measure them during integration.

Transformers are excellent when a GPU is available, but the rules require processing on the drone, and the Pi 5 has no usable GPU for them. A small YOLO fine-tuned from COCO weights is the right tool. When both stages run they are fused: a YOLO box confirmed by the blob stage gets higher confidence; a blob alone is down-weighted (x0.6); neural boxes must also pass the metre-size test.

4.3 Training data

tools/make_yolo_dataset.py renders labelled aerial images from the simulator: random altitude 20-45 m, random attitude and heading, ridge and dome tents of random size, random exposure and white balance, and every hard negative above. 1200 images (812 with a tent) are ready in datasets/tents_dataset.zip. Training runs on a free Colab GPU with tools/train_tent_yolo_colab.ipynb.

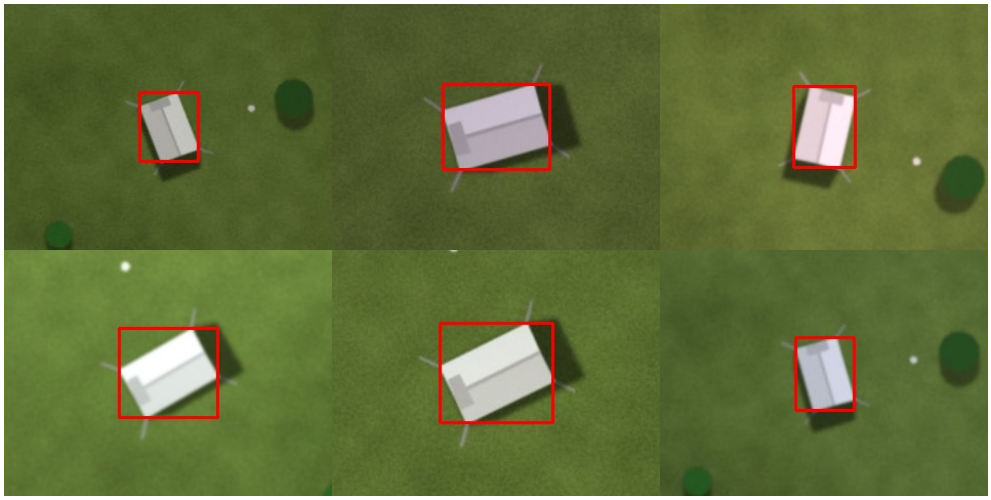


Figure 2 - samples of the generated YOLO training labels (red = label box).

Important: synthetic images teach geometry and negatives, but the final model must also be trained and, above all, **validated on real photos** from our test flights over a white tent.

5. Building the map

5.1 Placing each photo

The four image corners are projected to the ground with the camera model, the GPS position, the altitude and the roll/pitch/yaw. For flat ground, image-to-ground is exactly a **homography**, so tilt is corrected properly, not just rotated. The photo is first reduced to map resolution (5 cm/px), then warped onto a north-up canvas that grows as the drone flies.

5.2 Fixing GPS error between photos

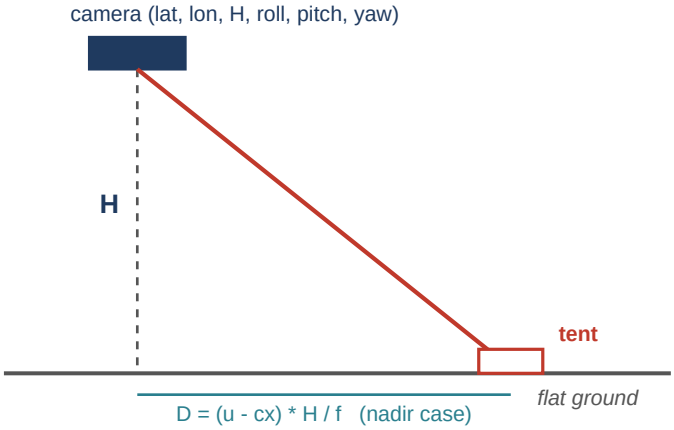
GPS is off by 1-2 m, so two neighbouring photos would not line up. In the overlap we measure the remaining shift with **phase correlation** (FFT based, fast, works on grass texture). We accept it only if confident and smaller than 3 m. Then we apply only 80 % of it. This **GPS anchoring** keeps the map following an average of GPS positions instead of slowly drifting. Measured in simulation over 206 photos:

Method	Map position error vs truth (mean / max)	Seam error between neighbours (mean / p90)
GPS only	1.05 m / 2.15 m	0.59 m / 1.01 m
Image alignment, no anchoring	1.52 m / 3.85 m (drifts)	0.36 m / 0.30 m
Alignment + 20 % GPS anchoring (used)	**0.94 m / 2.09 m**	**0.21 m / 0.31 m**

5.3 Seams and brightness

Each map pixel keeps the photo in which it was closest to the image centre: the most vertical view with the least lens distortion. This is the nadir seam rule used in professional orthomosaic software, and costs one integer comparison per pixel. Brightness is matched in the overlap. Memory use is about 5 bytes per map pixel (about 80 MB for 200 x 200 m at 5 cm). After a reboot the map is rebuilt from the saved thumbnails in seconds.

6. Tent GPS coordinates



general case (what the code does):

```
ray_cam = K^-1 [u, v, 1]
ray_NED = R(yaw,pitch,roll) R_mount ray_cam
P = (H / ray_down) * ray_NED
lat,lon = drone + (P_N, P_E) / radii
```

Figure 3 - pixel to ground. The simple formula from the plan is the special case of a perfectly level camera; the code uses the full rotation.

For each detection the box centre (and its 4 corners, for drawing) is turned into a ray, rotated by the drone attitude and the camera mounting, and intersected with flat ground H metres below (H = relative altitude + lens height). The resulting north/east offset is added to the drone position and converted to latitude/longitude with WGS-84 radii.

Fusing sightings and honest accuracy

- Sightings within 6 m are grouped; each group is scored by its best confidences and how many views agree. A one-off false detection cannot beat a tent seen 8 times.
- Position = inverse-variance weighted mean after removing outliers. Each sighting's variance comes from altitude x attitude error, distance x compass error, and altitude error x off-nadir angle.
- Random errors shrink with more views; **GPS bias does not**, so it is added back. We report a **95 % error radius** (2.45 sigma), not a fake precision.
- The box drawn on the map is snapped onto the tent's pixels in the mosaic, so it sits exactly on the tent. The printed coordinates remain the fused GPS estimate.

7. Test results (simulation)

The complete mission was run on 5 randomly generated worlds (different terrain, tent position/orientation, decoys and sun angle). Simulated sensor errors: GPS bias 1.2 m plus 0.25 m jitter, altitude bias 0.4 m, attitude noise 0.4 deg, compass bias 1.5 deg, wind sway 3 deg. The pipeline only ever sees the noisy measurements.

Seed (random world)	Photos	Sightings	Tent GPS error	Claimed 95 %	Map georef error	Box vs tent	Laptop ms/photo
7	192	8	1.29 m	3.8 m	1.33 m	0.0 m	123.4
11	192	8	0.72 m	4.0 m	1.08 m	0.0 m	117.9
23	191	7	0.75 m	3.9 m	0.79 m	0.0 m	119.2
42	190	7	1.32 m	3.8 m	0.91 m	0.0 m	114.7
99	193	7	1.24 m	3.8 m	1.43 m	0.0 m	114.9

Tent detected in 5/5 worlds, mean GPS error **1.06 m** (max 1.32 m), always inside the claimed 95 % radius. No decoy was ever reported as the tent. 13 automated unit tests pass (python -m pytest).



Figure 4 - final_map.png produced by the software (seed 42, downscaled). Note the decoys: the white-roof building and the white car are not boxed.

Close-up and files on the USB stick

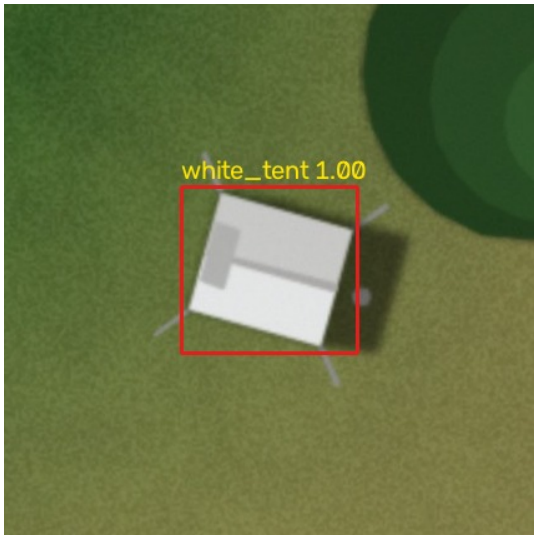


Figure 5 - tent_closeup.jpg: the onboard photo the tent position was taken from.

File	Content
final_map.png / .jpg	annotated map (required by the rules)
tent_coordinates.txt	lat/lon (decimal + DMS), 95 % accuracy, confidence, sightings
detections.json	fused estimate, every sighting, other candidates
image_metadata.csv	GPS / attitude / battery for every photo
mission_log.csv	telemetry log at 2 Hz
skyborne_map.kml + map_overlay.jpg	opens in Google Earth with the tent placemark
summary.json, MANIFEST.sha256	machine-readable summary; checksums verified after copying

Example tent_coordinates result: **33.6844976, 73.0478586**, confidence 1.00, +/- 3.8 m (95 %), 7 sightings.

8. How to run it

8.1 On the laptop (now)

```
cd Skyborne
.\venv\Scripts\activate
python -m pytest -q
python main.py sim --seed 5 --usb-dir results\_fake_usb
python main.py detect --image photo.jpg --alt 30
```

8.2 With ArduPilot SITL (next step)

Start SITL (WSL/Ubuntu: sim_vehicle.py -v ArduCopter --console --map --out=udp:<laptop-ip>:14551, or Mission Planner's Simulation tab with an extra UDP output), load the mission from tools/make_mission.py, then:

```
python main.py sitl --connect udpin:0.0.0.0:14551
```

The program anchors the synthetic world 90 m north of SITL home. Fly the survey in AUTO and land. On disarm it builds the map and copies it to a USB stick. This tests the real live loop: MAVLink thread, capture trigger, timing, finalize on disarm and export.

8.3 On the Raspberry Pi 5 (integration)

```
bash deploy/setup_pi.sh && sudo reboot
python main.py flight --new-session   # bench test
sudo systemctl enable --now skyborne   # autostart for flights
journalctl -u skyborne -f
```

Pi 5 pin	Pixhawk TELEM2	Note
GPIO14 (TXD, pin 8)	RX (pin 3)	3.3 V logic on both sides
GPIO15 (RXD, pin 10)	TX (pin 2)	crossed TX/RX
GND (pin 6)	GND (pin 6)	common ground is essential
5 V	- do NOT connect	power the Pi from its own 5 V 5 A BEC

Flight controller: SERIAL2_PROTOCOL = 2, SERIAL2_BAUD = 921 (see deploy/ardupilot_reference_params.txt, a reference for the safety pilot to check against the firmware docs, not a file to load blindly).

8.4 Competition-day sequence

- Power on: the Pi autostarts, waits for the heartbeat, and Mission Planner shows "SKY onboard ready".
- Arm and fly AUTO: laps, then the survey. The Pi photographs only over the search area; Mission Planner messages show progress ("SKY img 40 map 11135m2", "SKY tent? 0.98 ...").
- Land and disarm: the map is finalized automatically (or 150 s before the time limit, whichever comes first; or by creating data/FINALIZE).
- Insert the USB stick (it may already be in): files are copied, checksums verified, and "SKY: map saved to USB" appears. Hand the stick to the judges.
- If the Pi reboots mid-flight, the service restarts and **resumes the same session** from the saved photos and thumbnails.

9. Limitations, risks and next steps

Honest limitations (for the TDR)

- Flat-ground assumption: tent GPS and map assume the survey area is level with home. Slopes add error of about height difference x tan(off-nadir angle). Set mapping.ground_elevation_m or fit a rangefinder.
- The map is a GPS-based mosaic with image alignment, **not a survey-grade orthomosaic**. Absolute accuracy is limited by GPS (about 1-2 m). Tall objects lean (relief displacement).
- Results so far are from simulation. The blob thresholds and the YOLO model must be tuned and validated on real photos from our test flights.
- Not yet tested on hardware: Picamera2 timing, the real UART link, Pi 5 CPU timing, live SITL and the YOLO ONNX path (no trained model yet). These are covered by the integration checklist below.

Integration checklist (in order)

#	Step	Pass criterion
1	SITL run (main.py sitl) with a generated mission	map + tent on USB after disarm
2	Train YOLO on Colab, drop ONNX in models/	detector mode shows yolo+hsv
3	Pi 5 setup, camera test, calibrate_camera.py	RMS < 0.5 px
4	Pi <-> Pixhawk on the bench (props OFF)	telemetry at 10 Hz, STATUSTEXT in MP
5	Measure Pi processing time per photo	< 1.2 s (else proc_width 1280 / YOLO tiles)
6	Hand-carried walk test / low hover over a white tent	detections on real photos
7	Survey flight with a safety pilot	map complete, tent error < 3 m vs phone GPS
8	3 full timed rehearsals	USB handed over before the limit

Open questions for the organisers

- mission time limit (config: mission.time_limit_s, default 1200 s);
- size of the search area and allowed altitude (default 140 x 120 m at 30 m);
- accepted method for flight termination after 90 s (ArduPilot Advanced Failsafe or RC kill);
- whether a map in .jpeg is acceptable if the .png is very large (we write both).

Key settings you will most likely change

Setting (config/user_settings.json)	Default	Meaning
mission.time_limit_s / finalize_margin_s	1200 / 150	finish the map before time runs out
capture.survey_area	from make_mission.py	polygon where photos are taken
capture.forward_overlap	0.75	photo spacing (6.8 m at 30 m)
detection.tent_min/max_area_m2	2.5 / 22	expected tent size
detection.use_yolo	auto	on when models/tent_yolo.onnx exists
mapping.resolution_m	0.05	map pixel size (5 cm)
mavlink.connection / baud	/dev/ttyAMA0 / 921600	Pi UART to TELEM2